04_embeddings_rag

Local Embeddings + RAG Stack for HelixLLM / HelixCode on RTX 5090 (Blackwell), Rootless Podman

Scope: A fully local embeddings + retrieval + agent-memory stack running in **rootless Podman** on a single **RTX 5090 (32 GB GDDR7, Blackwell `sm_120`), Linux**, giving HelixLLM and the HelixCode coding-agent platform first-class RAG plus a durable memory subsystem ("HelixMemory"). **Author:** deep-research subagent (T1/main) **Date / access date for all citations:** 2026-07-06 **Revision:** 1 **Anti-bluff note (§11.4.6 / §11.4.99):** Every recommendation is cited with a URL + access date. Numbers taken from secondary blog/benchmark sources are marked as such; where a spec is well-established from a model's own release but a fetched source did not restate it, it is marked **UNCONFIRMED (verify against model card)**. No claim here is a runtime "PASS" — this is a design/selection document, not a test result.

---

0. TL;DR recommended stack

Layer	Recommendation	Why
General embedding	**Qwen3-Embedding-4B** (Apache-2.0), fallback **BGE-M3** (MIT)	Top-of-MTEB open family; MRL-truncatable dims; 4B fits comfortably in 32 GB alongside other services. BGE-M3 = MIT, dense+sparse+multi-vector in one model for hybrid.
Code embedding	**Qwen3-Embedding-4B** (same model, instruction-prompted for code) or **jina-code-embeddings-1.5b** (Qwen2.5-Coder backbone)	Strong CoIR/code-retrieval; one general+code model reduces VRAM. jina-code specialised for NL→code / code→code.

Layer	Recommendation	Why
Reranker	**bge-reranker-v2-m3** (Apache-2.0) default; **Qwen3-Reranker-0.6B/4B** when instruction-following matters	Safe multilingual default, cheap; Qwen3-Reranker for instructed relevance. Avoid Jina-reranker weights (non-commercial license) for self-host.
Serving (embed+rerank)	**HF Text-Embeddings-Inference (TEI)** primary; **Infinity** for multi-model / decoder-embedders / ColBERT	TEI = lowest latency, purpose-built; Infinity = one server hosting many embed+rerank+ColBERT models, OpenAI-compatible.
Vector DB	**Qdrant**	Single binary, rootless-friendly, native sparse+dense hybrid, fast filtered queries, strong metadata filtering for code RAG.
Agent memory (HelixMemory)	**Zep/Graphiti** (temporal knowledge graph) as the durable spine + **mem0** as the lightweight extraction layer; expose via **MCP**	Zep leads temporal reasoning (LongMemEval); mem0 = simplest extraction/ADD-UPDATE-DELETE. Both self-hostable.
Code-RAG orchestration	**LlamaIndex** (indexing/retrieval) + **tree-sitter cAST** chunking + code-graph (lumen/CodeGraph) fusion	AST-aware chunks + graph edges + vector recall + rerank.

1. Local embedding models (2026)

1.1 Landscape

The open-weight embedding families now match or beat commercial APIs on **MTEB**; the 2026 leaders for self-hosting are **Qwen3-Embedding**, **BGE-M3**, and the **Jina v3/v4/v5** line. (Innovative AIs, 2026-07-06; Milvus blog, 2026-07-06)

- **Qwen3-Embedding** is described as "the first local family that competes with commercial embedding APIs across the board," with the **8B** model at **MTEB-multilingual 70.58**, ranked **#1** on the multilingual leaderboard as of 2025-06-05; **Q4 quant ≈ 5 GB**. (Morph LLM, 2026-07-06; ailog, 2026-07-06)
- **BGE-M3** — MIT-licensed workhorse, **100+ languages**, **dense + sparse + multi-vector (ColBERT-style)** in one model, **8192-token** context, **1024-dim**, **568M** params, MTEB ≈ **63.2**. Best single model for hybrid retrieval. (ailog, 2026-07-06; Innovative AIs, 2026-07-06)

1.2 Model table

Model	Params	Dims (native / MRL)	Context	Multilingual	MTEB (multi)	License
Qwen3-Embedding-8B	8B	4096 (MRL-truncatable)	32K[1]	100+	**70.6**	Apache-
Qwen3-Embedding-4B	4B	2560 (MRL)[1]	32K[1]	100+	~69–70[1]	Apache-
Qwen3-Embedding-0.6B	0.6B	1024 (MRL)[1]	32K[1]	100+	mid-60s[1]	Apache-
BGE-M3	568M	1024	8192	100+	63.2	**MIT**
Jina-embeddings-v3	570M	1024 (MRL)	8192	89	62.8	CC-BY-N
Jina-embeddings-v4	3.8B	2048 (MRL)	multimodal	many	high	CC-BY-N
gte-Qwen2-7B-instruct	7B	3584	32K[1]	multi	~70[1]	Apache-

Model	Params	Dims (native / MRL)	Context	Multilingual	MTEB (multi)	License
Stella / stella-en-1.5B-v5	1.5B	up to 8192 (MRL)	512–8K[1]	English-centric	high (EN)	MIT[1]
nomic-embed-text-v1.5 / v2	137M / MoE	768 (MRL)	8192	v2 multilingual	mid	Apache-
mxbai-embed-large-v1	335M	1024	512	EN	mid	Apache-

[1] **UNCONFIRMED (verify against model card)** — spec is from the Qwen3-Embedding / gte / Stella releases (well-established) but not restated in the fetched secondary sources; confirm dims/context/MTEB per size before pinning. [2] Several Jina embedding/reranker **weights are CC-BY-NC** (non-commercial); for a commercial self-host use the API or pick Apache/MIT models. (futureagi, 2026-07-06)

1.3 Code-embedding quality (CoIR)

CoIR (Code Information Retrieval, ACL 2025 Main) is now the standard code-retrieval benchmark, with a leaderboard on MTEB. (CoIR GitHub, 2026-07-06) Relevant specialised code embedders:

- **jina-code-embeddings-0.5b / 1.5b** — built on **Qwen2.5-Coder** backbones; support NL→code and code→code directions. (arXiv 2508.21290, 2026-07-06)
- **Qwen3-Embedding-0.6B/4B/8B** — strong general + code; in-domain fine-tunes exist. (arXiv 2508.21290)
- **voyage-code-3** — top code retrieval (~67.1 reported) but **API-only, closed** → excluded for a local stack. (ailog)
- **CodeXEmbed / SFR-Embedding-Code** — generalist multilingual/multi-task code family (arXiv 2411.12644). (arXiv, 2026-07-06)
- **CORE-Bench** (arXiv 2606.11864) — newer agentic-coding retrieval benchmark worth tracking. (arXiv, 2026-07-06)

Recommendation: - **One general + one code**: **Qwen3-Embedding-4B** as the general model (it is also competitive on code via instruction prompting, keeping VRAM low), and **jina-code-embeddings-1.5b** as a code-specialised second index when NL↔code recall must be maximised. If VRAM/ops simplicity dominates, run **Qwen3-Embedding-4B only** and add a code index later. - Keep **BGE-M3** available specifically for its built-in **sparse** vectors to drive hybrid search cheaply.

2. Rerankers

Rerankers (cross-encoders) re-score the top-k from vector recall; they are the single highest-ROI RAG-quality lever. (futureagi, 2026-07-06)

Reranker	Size	License	Multilingual	Notes / when to use	Source
bge-reranker-v2-m3	568M (base 278M)	**Apache-2.0**	Yes (MIRACL-strong)	**Safe self-host default**; solid BEIR, cheap, well-tested in TEI/Infinity.	futureagi, agentset
Qwen3-Reranker-0.6B / 4B / 8B	0.6–8B	**Apache-2.0**	Yes	Use when you want **instructed relevance** (tell it what "relevant" means) or multilingual on-prem; 0.6B beats bge on instruction-following.	futureagi
jina-reranker-v3	0.6B	**CC-BY-NC²**	Yes	SOTA BEIR **61.94 nDCG@10**, +5.43% vs bge at same 0.6B, 10× smaller than listwise; but **non-commercial weights → API only**.	jina, arXiv 2509.25085
	~2B	Apache-2.0	Yes		futureagi

Reranker	Size	License	Multilingual	Notes / when to use	Source
mxbai-rerank-large-v2				Apache alternative with managed-API option; larger footprint.	
Cohere Rerank 4	—	Closed API	Broad	Managed accuracy, not local.	futureagi

When to use: always rerank the top ~50–100 vector hits down to ~8–20 for the LLM. For HelixCode: **bge-reranker-v2-m3** as default; switch to **Qwen3-Reranker-4B** for hard code queries where you pass an instruction ("rank by which snippet implements the described API"). Do **not** ship jina-reranker weights in a commercial product (license); use it only via API for evaluation.

---

3. Vector database (self-hosted, rootless Podman)

DB	Fit for this deployment	Verdict
Qdrant	Single Rust binary / one container image, minimal ops, native **sparse+dense hybrid**, integrated payload (metadata) index → fast **filtered** queries (path, language, repo, symbol). Rootless-podman-friendly.	**RECOMMENDED**
Milvus	Billion-scale, K8s-native distributed; heavier (etcd/minio/pulsar deps) — overkill for one 5090.	Only if you outgrow ~50–100M vectors.
pgvector	Best **if already on Postgres**; v0.9 (early 2026) added sparse-vector + speed;	Strong "boring" choice if HelixMemory/metadata already live in PG.

DB	Fit for this deployment	Verdict
	comfortable to ~50M vectors.	
LanceDB	Embedded/edge, great for desktop/DS; multi-process concurrency limits, smaller community, cloud beta.	Good for a single-agent local index; weaker for a shared service.
Weaviate	Built-in hybrid + vectorizers; more moving parts than Qdrant.	Pick if you want built-in vectorizer modules.
Chroma	Prototyping.	Dev only.

Sources: CallSphere benchmarks 2026 (2026-07-06); Firecrawl best vector DBs (2026-07-06); DEV: pgvector vs Qdrant vs Milvus (2026-07-06); Zilliz Qdrant vs LanceDB (2026-07-06).

Recommendation: Qdrant. "Qdrant is the better default for most production RAG pipelines in 2026 — lighter, faster on filtered queries, simpler to operate." It supports **named vectors** (store dense Qwen3 + sparse BGE-M3 + code-model vectors per point) and **hybrid fusion (RRF)** natively — exactly what a code-RAG index needs. If Postgres is already central to Helix, **pgvector** is the acceptable alternative to avoid a second datastore.

---

4. Code-RAG framework & patterns

4.1 Chunking source trees — AST-aware

Naive fixed-width splitting breaks functions mid-body and strips class/import context. Use **structure-aware (AST) chunking**:

- **cAST** (arXiv 2506.15655) — recursively splits large AST nodes and merges siblings under a size budget, preserving syntactic integrity; validated gains over fixed-size chunking on retrieval and RAG; usable as a plug-and-play chunker for agents. (arXiv, 2026-07-06)
- **tree-sitter** parses to an AST; extract semantic entities (functions, methods, classes, interfaces, types, imports). (Supermemory code-chunk, 2026-07-06; code-chunk GitHub)
- **Contextualized chunks** — prepend scope/file-path/signature/dependencies to each chunk before embedding (big recall win). (Supermemory, 2026-07-06)

4.2 Code graph + embeddings hybrid (complementing lumen / CodeGraph)

Vector recall answers "what is semantically similar"; a **code graph** answers "what actually calls/imports/defines this." Fuse both:

- Graph-guided repo-level code agents (**GraphCodeAgent**, arXiv 2504.10046; **KG-based repo code gen**, arXiv 2505.14394) model the repo as a graph (call/def/import edges) and retrieve by syntactic + semantic + graph queries. (arXiv 2504.10046, 2026-07-06; arXiv 2505.14394, 2026-07-06)
- **How it complements CodeGraph/lumen:** lumen already provides a local SQLite semantic code-knowledge-graph over MCP (`semantic_search`, symbol resolution). The embeddings+vector layer here is the **recall breadth** engine (fuzzy NL→code, cross-file semantic similarity, doc/comment retrieval) while lumen/CodeGraph supplies **precise structural edges** (definition, callers, transitive deps). The agent's retriever should **union** both: (1) vector+rerank hits from Qdrant, (2) graph-expanded neighbours from lumen, then rerank the merged set. This is the §11.4.78/§11.4.79 code-intelligence layer + the RAG layer working together, not competing.

4.3 GraphRAG relevance

GraphRAG (graph index + community summaries) helps for **whole-repo "how does subsystem X work"** questions that need aggregation across many files; it is heavier to build/maintain. Use it selectively for architecture-level Q&A, not for every retrieval. LlamaIndex has property-graph + GraphRAG pipelines. (LlamaIndex GraphRAG, 2026-07-06)

4.4 Framework choice

- **LlamaIndex** — deepest indexing/retrieval abstractions (node parsers, property graph, custom query engines) → **primary** for the code-RAG pipeline. (Contra, 2026-07-06)
- **Haystack** — component/DAG pipelines (retrievers, rankers, readers) → good if you want explicit pipeline graphs.
- **Custom thin retriever** — many coding agents ship a small custom retriever calling TEI + Qdrant + reranker directly; lowest dependency surface. Recommended if Helix wants tight control (§11.4.28 decoupling).

4.5 Context assembly for agents

1. Query → embed (Qwen3-Embedding) + sparse (BGE-M3) → **Qdrant hybrid (RRF)** top-k (~50–100).
2. **Graph-expand** top hits via lumen/CodeGraph (callers/callees/defs).
3. **Rerank** merged candidates (bge-reranker-v2-m3) → top ~8–20.

4. Assemble with **contextual headers** (file path, symbol, language) + dedupe + token budget.
5. Log retrieval evidence for anti-bluff (§11.4.5/§11.4.69).

---

5. HelixMemory — durable local agent memory

System	Model	Local?	Strength	Source
Zep / Graphiti	Temporal knowledge graph; timestamps every fact ("when it was true")	Yes (self-host; needs a graph store)	**Best temporal reasoning** — 63.8% LongMemEval vs mem0 49.0%	MCP.Directory, 2026-07-06, rohitraj, 2026-07-06
mem0	Extraction layer: per-message-pair salient-fact ADD/UPDATE/DELETE/NOOP; vector+graph+KV	Yes (self-host)	**Simplest to bolt on**, 47K+ stars, biggest community	MCP.Directory, particula
Letta (MemGPT)	Runtime where the agent **is** its memory: self-edited core block + archival store	Yes	Stateful long-running agents-as-services	MCP.Directory
Cognee	Graph+vector memory pipeline	Yes	ECL (extract-cognify-load)	atlan, 2026-07-06
OMEGA	Fully local, SQLite + ONNX embeddings, **zero external deps**, MCP-native, AES-256 at rest	Yes	Simplest zero-dep local + MCP for coding agents	MCP.Directory

Episodic vs semantic: Letta = self-managed core (semantic, agent-editable) + archival (episodic search); mem0 = distilled semantic facts with mutation; Zep = temporal graph (episodic events → semantic facts with validity intervals).

Recommendation for HelixMemory: - **Primary durable store: Zep/Graphiti** — its temporal graph is the right substrate for a coding platform where facts change

over time (a function's signature, a config value, "the bug we fixed last week"). This aligns with Helix's SQLite-SSoT + history discipline (§11.4.93/§11.4.95) at the *agent* layer. - **Extraction/convenience layer: mem0** — 5-line integration for per-session fact capture; or **OMEGA** if zero-dependency + MCP-native local is preferred for the CLI coding agents. - **Expose via MCP**: run the memory server as an MCP server (`add`/`search`/`update`) so Claude Code / OpenCode / Qwen / Crush all reach it uniformly (mirrors §11.4.78 CodeGraph MCP wiring). Keep it **project-agnostic** (§11.4.28) — the consumer registers namespaces at runtime.

Caveat (§11.4.6): LongMemEval numbers above are from secondary blogs quoting the Zep paper; verify against the primary Zep/Graphiti evaluation before treating 63.8 vs 49.0 as canonical.

---

6. Serving + rootless Podman

6.1 Serving engines

- **TEI (HF text-embeddings-inference)** — purpose-built for embeddings/rerankers (short fixed inputs, no KV cache, vector outputs); **lowest latency**, serves embeddings **and** rerankers. **Primary** engine. (Spheron, 2026-07-06; kunwar.page ch.49, 2026-07-06)
- **Infinity** — one server hosting **many** models at once (embeddings + rerankers + ColBERT/ColPali + CLIP), **OpenAI-compatible** API, backends **PyTorch** / **ONNX** / **TensorRT** / **CTranslate2**; explicitly supports BGE + Jina (embed & rerank) and decoder Qwen2 embedders. Use when you need **multi-model** or ColBERT/multi-vector. (Infinity GitHub, 2026-07-06)
- **vLLM** — highest **throughput** for large decoder-based embedders and for co-serving the generation LLM; v0.11.0 **natively supports Blackwell/RTX 5090** (NVFP4/CUTLASS). Use for big-batch offline indexing or when the embedder is a 7B+ decoder. (Snowflake/vLLM embeddings, 2026-07-06; allenkuo Blackwell, 2026-07-06)

Throughput/latency summary from sources: **vLLM** leads raw throughput; **TEI** leads latency; specialised servers (Arctic/BEI) beat both on throughput. (Snowflake, 2026-07-06; Baseten BEI, 2026-07-06)

6.2 RTX 5090 / Blackwell notes

- **32 GB GDDR7 @ ~1.79 TB/s**; runs 32B-class at Q4 on one card. (MadCoolStuff, 2026-07-06)
- Needs **recent CUDA (12.8+/13.x)** + **driver 580+** for full Blackwell/FP8/FP4 kernels; some quant libs needed Blackwell patches through early 2026 (mostly closed by mid-2026). (arXiv 2601.09527, 2026-07-06; allenkuo, 2026-07-06)

- **VRAM budgeting on one 5090:** the generation LLM will dominate. Keep embedders small: Qwen3-Embedding-4B (~8 GB) + bge-reranker-v2-m3 (~1 GB) + BGE-M3 (~2 GB) ≈ **11 GB** for the retrieval tier, leaving ~20 GB for a quantized code LLM. If co-hosting a large LLM, drop to **Qwen3-Embedding-0.6B** (~1.5 GB) or move indexing to CPU/off-peak.

6.3 Rootless Podman + GPU (CDI)

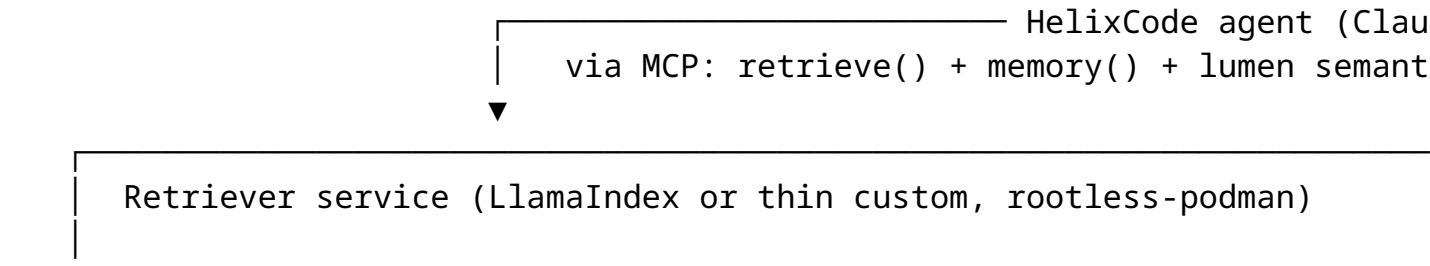
The correct 2026 path is **CDI (Container Device Interface)** via the NVIDIA Container Toolkit — the recommended, rootless-friendly mechanism. (NVIDIA CDI docs, 2026-07-06; Podman Desktop GPU, 2026-07-06; OneUptime, 2026-07-06)

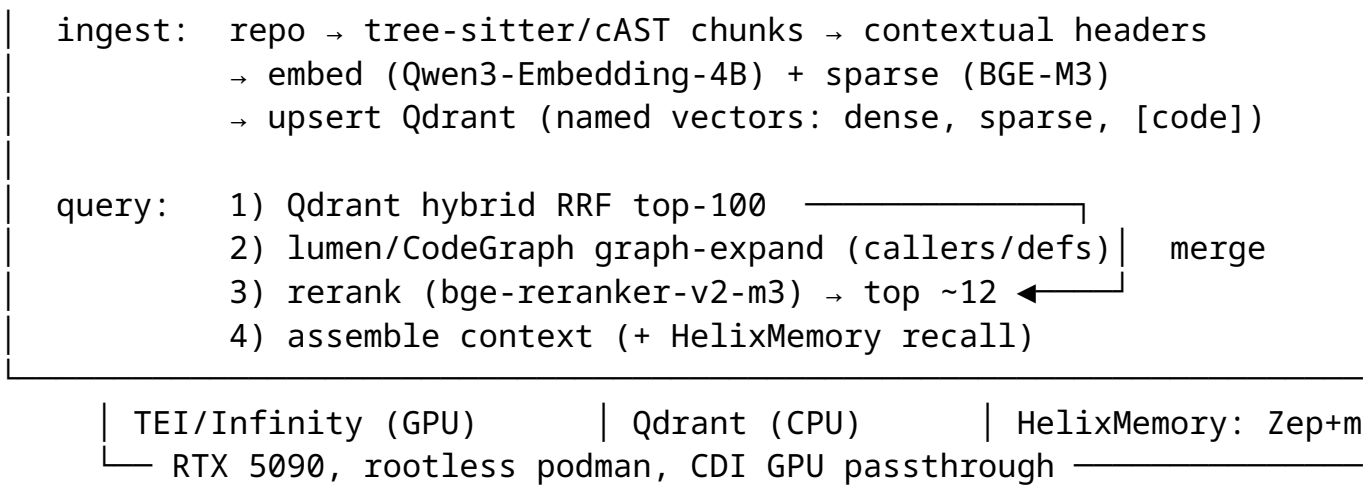
```
# 1. Generate a CDI spec (rootless: into the user config dir)
mkdir -p ~/.config/cdi
nvidia-ctk cdi generate --output=$HOME/.config/cdi/nvidia.yaml
#    (system-wide alternative: sudo nvidia-ctk cdi generate --
        output=/etc/cdi/nvidia.yaml)
#    Toolkit >= v1.18.0 auto-refreshes via the nvidia-cdi-refresh
        systemd unit.

# 2. Run a GPU container rootless (Podman >= 4.1 supports --
        device CDI refs)
podman run --rm \
  --device nvidia.com/gpu=all \
  --security-opt=label=disable \
  ghcr.io/huggingface/text-embeddings-inference:latest \
  --model-id Qwen/Qwen3-Embedding-4B
```

- Aligns with Helix **§11.4.161 rootless-container mandate** and **§11.4.76 containers submodule** — drive boot/health through the containers submodule's `pkg/boot`/`pkg/compose`, not ad-hoc `podman run`, in the real deployment.
- Per-service containers: `tei-embed`, `tei-rerank` (or one `infinity` container hosting both + code embedder), `qdrant`, `helixmemory` (zep/mem0), on a shared rootless pod network. Qdrant needs **no GPU**.

---

7. Reference architecture sketch (code-RAG)





Data flow: AST chunk → contextual-embed → hybrid store → hybrid recall → graph-fuse → rerank → assemble → agent. Memory writes (episodic facts, resolved bugs, decisions) go to Zep/Graphiti; memory reads join the context assembly step.

---

8. Top 3 risks

1. **Blackwell / `sm_120` toolchain drift.** Quant kernels, TEI/Infinity/vLLM builds, and some embedders needed CUDA 12.8+/13.x + driver 580+ and Blackwell patches; a mismatched image silently falls back to slow paths or fails to load FP8/FP4. **Mitigation:** pin CUDA 13.x base images, driver ≥580, test each container's `nvidia-smi` + a real embed call before trusting it (§11.4.108 runtime signature). (arXiv 2601.09527)
2. **VRAM contention on a single 32 GB card.** A large code LLM + embedder + reranker + big-batch indexing can OOM. **Mitigation:** size embedders down (4B or 0.6B), run heavy re-indexing off-peak or on CPU, cap batch sizes; consider a second index pass rather than co-resident everything.
3. **License traps + benchmark over-trust.** Several **Jina embedding/reranker weights are CC-BY-NC** (not for commercial self-host); and headline MTEB/LongMemEval numbers come from secondary blogs. **Mitigation:** ship only Apache-2.0/MIT weights (Qwen3, BGE, Stella, nomic, mxbai); verify every headline number against the model card / primary paper before pinning (§11.4.6/§11.4.99). Secondary risks: rerank latency added to every query (budget it), and code-chunking edge cases (giant files, generated code) — validate cAST output on the real repo.

Sources verified 2026-07-06

- Innovative AIs — Best Embedding Models for RAG 2026 — https://innovativeais.com/blog/best-embedding-models-for-rag-in-2026

- Morph LLM — Best Ollama Embedding Models 2026 (MTEB/VRAM/dims) — https://www.morphllm.com/ollama-embedding-models
- Milvus blog — Best Embedding Model for RAG 2026 (10 models) — https://milvus.io/blog/choose-embedding-model-rag-2026.md
- ailog — Embedding Models 2026 benchmark & comparison — https://app.ailog.fr/en/blog/news/embedding-models-2026
- CoIR — Comprehensive Benchmark for Code Information Retrieval (ACL 2025) — https://github.com/coir-team/coir
- arXiv 2508.21290 — Efficient Code Embeddings from Code Generation Models (jina-code) — https://arxiv.org/html/2508.21290v1
- arXiv 2411.12644 — CodeXEmbed — https://arxiv.org/pdf/2411.12644
- arXiv 2606.11864 — CORE-Bench (agentic code retrieval) — https://arxiv.org/pdf/2606.11864
- futureagi — Best Rerankers for RAG 2026 — https://futureagi.com/blog/best-rerankers-for-rag-2026/
- jina.ai — jina-reranker-v3 model page — https://jina.ai/models/jina-reranker-v3/
- arXiv 2509.25085 — jina-reranker-v3 — https://arxiv.org/html/2509.25085v2
- agentset — BGE Reranker v2-m3 vs Jina Reranker v2 — https://agentset.ai/rerankers/compare/baaibge-reranker-v2-m3-vs-jina-reranker-v2-base-multilingual
- CallSphere — Vector DB Benchmarks 2026 (pgvector/Qdrant/Weaviate/Milvus/LanceDB) — https://callsphere.ai/blog/vector-database-benchmarks-2026-pgvector-qdrant-weaviate-milvus-lancedb
- Firecrawl — Best Vector Databases 2026 — https://www.firecrawl.dev/blog/best-vector-databases
- DEV — pgvector vs Qdrant vs Milvus 2026 — https://dev.to/linou518/choosing-the-foundation-for-your-rag-system-pgvector-vs-qdrant-vs-milvus-2026-4i5o
- Zilliz — Qdrant vs LanceDB — https://zilliz.com/comparison/qdrant-vs-lancedb
- Spheron — Self-host Embeddings & Rerankers: TEI on GPU (2026) — https://www.spheron.network/blog/self-host-embedding-reranker-tei-gpu-cloud/
- kunwar.page — Ch.49 TEI for embeddings & rerankers in production — https://www.kunwar.page/chapter/049-tei-for-embeddings-and-rerankers-in-production
- Infinity (michaelfeil) — embedding/reranker/ColBERT server — https://github.com/michaelfeil/infinity
- Snowflake/vLLM — Scaling vLLM for Embeddings (16x) — https://www.snowflake.com/en/engineering-blog/embedding-inference-arctic-16x-faster/
- Baseten — How we built BEI (high-throughput embed/rerank) — https://www.baseten.co/blog/how-we-built-bei-high-throughput-embedding-inference/

- cAST — Structural Chunking via AST (arXiv 2506.15655) — https://arxiv.org/abs/2506.15655
- Supermemory — code-chunk AST-aware chunking — https://supermemory.ai/blog/building-code-chunk-ast-aware-code-chunking/ ; https://github.com/supermemoryai/code-chunk
- arXiv 2504.10046 — GraphCodeAgent — https://arxiv.org/pdf/2504.10046
- arXiv 2505.14394 — Knowledge-Graph repo-level code gen — https://arxiv.org/pdf/2505.14394
- LlamaIndex GraphRAG pipeline — https://medium.com/@tuhinsharma121/beyond-rag-building-a-graphrag-pipeline-with-llamaindex-for-smarter-structured-retrieval-3e5489b0062c
- Contra — LlamaIndex vs Haystack 2026 — https://contracollective.com/blog/llamaindex-vs-haystack-rag-pipeline-2026
- MCP.Directory — Mem0 vs Letta vs Zep vs Cognee 2026 — https://mcp.directory/blog/mem0-vs-letta-vs-zep-vs-cognee-2026
- rohitraj — Open-source AI agent memory: Mem0 vs Zep vs Letta 2026 — https://rohitraj.tech/en/notes/open-source-ai-agent-memory-mem0-vs-zep-letta-2026
- particula — Agent memory frameworks tested — https://particula.tech/blog/agent-memory-frameworks-tested-mem0-zep-letta-cognee-2026
- atlan — Best AI agent memory frameworks 2026 — https://atlan.com/know/best-ai-agent-memory-frameworks-2026/
- NVIDIA — CDI support (Container Toolkit) — https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/cdi-support.html
- Podman Desktop — GPU container access — https://podman-desktop.io/docs/podman/gpu
- OneUptime — Run NVIDIA GPU containers with Podman (2026-03) — https://oneuptime.com/blog/post/2026-03-18-run-nvidia-gpu-containers-podman/view
- MadCoolStuff — RTX 5090 local inference — https://madcoolstuff.com/reviews/rtx-5090-local-inference
- arXiv 2601.09527 — Private LLM Inference on Consumer Blackwell GPUs — https://arxiv.org/html/2601.09527v1
- allenkuo — vLLM or Ollama on Blackwell — https://allenkuo.medium.com/vllm-or-ollama-on-blackwell-benchmarks-landmines-and-what-agents-actually-need-5dc539bb28ef